CMP653 Project Proposal

A Differentially Private SQL Middleware for Repeated Aggregate Queries

Refik Can Öztaş
Hacettepe University
N25142279

Abstract

This proposal studies differential privacy in database systems, with a particular focus on practical SQL query processing. The document consists of two parts. The first part is a reaction paper that reviews recent work on differentially private relational query evaluation, including join sensitivity, foreign-key-aware mechanisms, multi-query budget management, and practical DP-SQL system design. The second part presents a project proposal for a lightweight differentially private SQL middleware targeting repeated aggregate queries over a relational benchmark. The proposed system introduces explicit privacy budget accounting and a workload-aware allocation policy within a scope appropriate for a project. The evaluation will include mathematical analysis, utility measurements, and system overhead.

1 Part I: Reaction Paper

1.1 Motivation and Context

Differential privacy provides a formal framework for answering statistical queries while limiting the contribution of individual records. In relational database systems, however, its practical use remains challenging. Queries involving joins may have very high sensitivity, which in turn requires the addition of substantial noise and often leads to poor utility. In addition, analytical workloads usually consist of multiple related queries rather than isolated single-query executions. In such settings, privacy budget management becomes a central systems issue, since naive composition can consume the budget too quickly.

Foundational systems and frameworks such as PINQ [5], practical SQL-oriented DP mechanisms [6], and PrivateSQL [7] established the earlier direction of privacy-aware query processing. More recent database research has moved beyond single-query mechanisms and increasingly focuses on relational semantics, workload-aware accounting, and practical system design. The papers by Dong and Yi [1], Dong et al. [2], Dong et al. [3], and Yu et al. [4] collectively illustrate this development. They address four connected challenges: join sensitivity, exploitation of schema constraints, repeated-query budget management, and implementation in a usable SQL processing framework.

1.2 Paper 1: Residual Sensitivity for Differentially Private Multi-Way Joins

Dong and Yi [1] study the problem of answering multi-way join queries under differential privacy. The main challenge is that global sensitivity for such queries is often excessively large, since the effect of a single tuple may propagate through several join paths. To address this issue, the paper introduces a residual sensitivity approach that provides a tighter, data-dependent bound for join-count queries.

The main contribution of the paper is the observation that the privacy cost of a query is closely tied to the structure of the relational expression itself. Instead of treating differential privacy as a post-processing step on top of SQL, the authors show that relational structure must be incorporated directly into the privacy analysis. This makes the paper highly relevant for database systems research, since it connects differential privacy to relational algebra and query evaluation.

A limitation of the work is that it focuses on a restricted class of relational queries and relies on sophisticated sensitivity analysis that may be difficult to integrate into a lightweight prototype. Nevertheless, it establishes an important baseline for understanding why joins are one of the main obstacles in practical DP-SQL systems.

1.3 Paper 2: R2T and Foreign-Key-Aware Differential Privacy

Dong et al. [2] extend the discussion from general join sensitivity to more realistic relational schemas that contain foreign keys and self-joins. The paper proposes an instance-optimal truncation mechanism for differentially private query evaluation under foreign-key constraints. The main idea is that schema-level information can be used to improve privacy-utility tradeoffs by avoiding unnecessarily conservative sensitivity bounds.

The paper makes two important contributions. First, it shows that foreign-key relationships are not only useful for schema design and query optimization, but also for privacy mechanism design. Second, it demonstrates that practical database constraints can significantly affect the amount of noise required for differential privacy. This is especially relevant for database systems, where schema information is already available and should not be ignored by a privacy-aware query processor.

One limitation is that the method still operates under a restricted query model and depends on assumptions about schema structure. Even so, the paper is valuable because it moves the literature toward more realistic relational settings and suggests that privacy-aware query processing should make use of available database metadata whenever possible.

1.4 Paper 3: Better than Composition

Dong, Sun, and Yi [3] address a different but closely related problem: repeated relational queries under a fixed privacy budget. The standard approach in differential privacy is to allocate budget independently to each query and rely on composition to obtain the total privacy cost. While mathematically correct, this often leads to poor accuracy when a workload contains many related queries.

The authors show that, under suitable conditions, multiple relational queries can be answered with tighter error guarantees than those obtained by naive composition. The main importance of

this paper is that it shifts attention from single-query mechanisms to workload-level privacy management. This is highly relevant for practical database systems such as dashboards, reports, and exploratory analytics, where analysts issue sequences of related aggregate queries.

The main limitation is that translating such workload-level theory into a practical system is not straightforward. A working system would need explicit budget accounting, stateful query management, and potentially query-template tracking. Nonetheless, the paper provides a strong conceptual basis for studying privacy budget allocation as a database systems problem rather than only a theoretical privacy problem.

1.5 Paper 4: DOP-SQL as a Practical System Direction

Yu et al. [4] present DOP-SQL, a general-purpose and extensible private SQL system built on top of PostgreSQL. The paper focuses on practical system design, including SQL parsing, mechanism selection, and query execution support for private SQL processing. Compared with the previous papers, DOP-SQL places less emphasis on a single new privacy theorem and more emphasis on integration into an actual DBMS-oriented architecture.

This paper is important because it demonstrates that differential privacy research in databases has progressed beyond isolated mechanisms toward end-to-end system design. In other words, the field is moving from theoretical building blocks to software infrastructure that can support private query answering in practice.

At the same time, the paper also highlights how difficult it is to provide broad SQL support under differential privacy. Although the system is general-purpose in direction, the practical scope must still be constrained. For a course project, this suggests that a smaller but well-defined SQL subset is a more realistic and scientifically sound target than an attempt to support arbitrary SQL.

1.6 Synthesis and Critique

Taken together, these papers suggest that practical differentially private query processing is shaped by two interacting factors: the *relational structure* of queries and the *management of privacy budget* across workloads. The first two papers focus on the relational side of the problem. Residual Sensitivity [1] shows that joins are one of the main reasons why differentially private SQL processing is difficult, because sensitivity can grow rapidly with join structure. R2T [2] further shows that schema information, especially foreign-key constraints, can be used to improve utility. These papers therefore establish an important background: DP-SQL is not only about adding noise to answers, but also about understanding query structure and database semantics.

The latter two papers are more directly aligned with the project proposed in this document. Better than Composition [3] highlights that repeated-query workloads create a separate systems problem: even when each query can be answered privately in isolation, naive composition may consume the privacy budget too quickly. DOP-SQL [4] demonstrates that these ideas must ultimately be implemented in a practical SQL processing framework with parsing, mechanism selection, and execution support. Together, these two papers motivate a workload-aware and system-oriented direction rather than a purely mechanism-oriented one.

At the same time, the literature also reveals several limitations. First, most existing methods still operate on restricted subsets of SQL, especially counting queries or SPJA-style workloads. Second, supporting joins, foreign keys, and richer SQL semantics often requires more complex analysis than is realistic for a project. Third, workload-level privacy accounting is conceptually appealing, but translating it into a practical interface requires explicit state management, privacy ledgers, and careful control of query scope. For this reason, a scoped project that focuses on repeated aggregate workloads is more realistic than an attempt to implement a fully general private SQL engine.

1.7 Brainstorming and Research Direction

The reviewed literature suggests a natural research direction: rather than attempting to solve the full problem of general SQL under differential privacy, it is more practical to focus on a narrower but still meaningful systems problem. In particular, repeated aggregate workloads appear to be a promising target. Such workloads are common in dashboards, reporting systems, and exploratory analytics, where analysts issue many related aggregate queries over the same dataset. In these settings, the main challenge is not only how to privatize a single query, but also how to manage the privacy budget across a sequence of related queries.

The reviewed literature therefore motivates two broad directions: broader support for complex relational query classes, and smarter privacy budget management for repeated analytical workloads. This project deliberately chooses the second direction in order to produce a scoped and implementable systems contribution. More specifically, the project will build a lightweight DP-SQL middleware for repeated aggregate queries and study whether explicit workload-level accounting can outperform naive per-query budget splitting. The emphasis will be on a small but practical SQL subset, such as `COUNT`, `SUM`, `AVG`, `GROUP BY`, and simple `WHERE` clauses.

Several concrete research questions follow from this direction. First, can a workload-level budget ledger improve utility compared with allocating the same amount of privacy budget independently to each query? Second, can repeated query templates be identified and handled more efficiently through template-aware accounting? Third, if time permits, can a lightweight join-aware heuristic improve a very limited subset of join-count queries without significantly increasing system complexity? The first two questions form the core of the proposed project, while the third is an optional extension motivated by the earlier join-focused papers.

2 Part II: Formal Proposal

2.1 Problem Statement and Research Questions

This project proposes a lightweight differentially private SQL middleware for repeated aggregate query workloads. The central problem is that when multiple aggregate queries are executed over the same dataset, naive privacy budget splitting may lead to inefficient use of the total budget and reduced utility. The project therefore investigates whether a workload-aware middleware can manage the privacy budget more effectively while maintaining acceptable system overhead.

The project is guided by the following research questions:

(1) How should a fixed privacy budget be allocated across a workload of repeated aggregate queries?
(2) Can a workload-level budget ledger reduce waste compared with stateless per-query composition?
(3) What is the trade-off between utility and system overhead under workload-level accounting?
(4) If time permits, can a limited join-aware heuristic improve the utility of a small subset of join-count queries?

The scope is intentionally restricted. The proposed middleware will support basic aggregate queries such as `COUNT`, `SUM`, and `AVG`, together with `GROUP BY` and simple `WHERE` clauses. Tuple-level differential privacy will be assumed, with bounded contribution for sums and averages.

2.2 Data and Workload

The project will use the TPC-H benchmark [8], initially at a small scale factor such as SF=0.1 or SF=1. TPC-H is appropriate because it is a standard relational benchmark that contains both single-table aggregates and join-based analytical queries. In addition, its schema and workload style are well suited to studying repeated query execution patterns.

A repeated-query workload will be generated by varying predicates, groupings, and simple query templates. This workload will simulate exploratory analytical use, where similar aggregate queries are executed repeatedly over the same data. Such a setting is appropriate for evaluating workload-level privacy budget management.

2.3 Algorithms, Techniques, and System Design

The prototype will be implemented in Python as a middleware layer on top of PostgreSQL. Its main components will be:

(1) **SQL parser and validator:** checks whether the incoming query belongs to the supported subset;
(2) **Query analyzer:** identifies the aggregate type and determines the required sensitivity assumptions;
(3) **DP mechanism layer:** initially applies the Laplace mechanism;
(4) **Budget ledger:** records privacy budget consumption across the workload;
(5) **Template-aware handling:** reuses workload-level information for repeated exact or template-matching queries, if feasible.

The baseline private approach will allocate privacy budget independently to each query. The proposed method will instead use a workload-level accounting strategy. If time permits, a limited join-aware variant will also be explored for a restricted set of join-count queries, drawing inspiration from recent work on join sensitivity and foreign-key-aware truncation [1, 2].

The mathematical analysis will include query sensitivity for the supported aggregate operators, bounded-contribution assumptions for `SUM` and `AVG`, and privacy accounting under repeated-query workloads. For example, `COUNT` queries will use unit sensitivity under tuple-level privacy, while bounded domains will be required for other aggregate types.

2.4 Evaluation Plan

The evaluation will compare the following three approaches:

- **Baseline 1 (Exact SQL):** standard non-private SQL query execution;
- **Baseline 2 (Naive DP):** independent privacy budget consumption for every query;
- **Proposed Middleware:** workload-level budget ledger with explicit allocation policy.

The main hypothesis is that workload-level budget accounting will achieve lower query error than naive per-query allocation under the same total privacy budget, while introducing acceptable middleware overhead.

Success will be measured along three dimensions:

(1) **Utility:** absolute error, relative error, MAE, and RMSE compared with exact query results;
(2) **Budget efficiency:** number of queries answered within a fixed total privacy budget;
(3) **System performance:** latency overhead, including p95 latency for repeated workloads.

The experimental evaluation will compare error and overhead under different workload sizes and allocation strategies.

2.5 Expected Contribution

The expected outcome is a scoped and reproducible study of privacy budget management in repeated aggregate SQL workloads. The contribution is not intended to be a full private DBMS, but rather a focused middleware prototype that investigates one concrete systems question: whether workload-aware budget accounting provides a practical improvement over naive per-query composition for repeated aggregate queries.

The originality of the proposed work lies in a scoped architectural integration rather than in a new privacy theorem. Specifically, the project combines workload-level privacy budget accounting, template-aware handling of repeated queries, and lightweight SQL middleware execution in a single experimental framework for repeated aggregate workloads. It also provides a systematic comparative evaluation against naive per-query composition. In this sense, the project contributes both a novel architectural integration and a rigorous comparative performance analysis for privacy-aware SQL processing.

2.6 Summary

The proposed project can be summarized in terms of its problem setting, research direction, main focus, and implementation plan.

The Problem: Repeated aggregate SQL workloads under differential privacy may suffer from poor utility when privacy budget is consumed independently for each query. In practical analytical settings such as dashboards, reports, and exploratory workloads, naive per-query composition can lead to inefficient budget use and reduced answer quality.

Directions for Solution and Research: This project will study workload-aware privacy budget management for repeated aggregate SQL queries. In particular, it will investigate whether workload-level accounting and template-aware handling of repeated queries

can improve utility and budget efficiency compared with naive per-query allocation.

Research Focus: The main research focus is to evaluate whether a lightweight differentially private SQL middleware can provide better utility–budget trade-offs than stateless per-query composition, while maintaining acceptable system overhead in a practical DBMS setting.

Plan of Work:

- **Phase 1 (Weeks 4–6):** Finalize the literature review, set up PostgreSQL and the TPC-H benchmark, and implement the exact SQL baseline together with a naive differentially private baseline.
- **Phase 2 (Weeks 7–9):** Develop the middleware prototype, including SQL parsing and validation, supported aggregate query handling, and a workload-level privacy budget ledger.
- **Phase 3 (Weeks 10–12):** Implement the workload-aware allocation strategy and evaluate utility, budget efficiency, and query latency on repeated aggregate workloads.
- **Phase 4 (Weeks 13–14):** Analyze results, compare against baselines, and prepare the final paper in ACM conference format.

References

[1] Wei Dong and Ke Yi. 2021. Residual Sensitivity for Differentially Private Multi-Way Joins. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. ACM, 432–445. https://dl.acm.org/doi/10.1145/3448016.3452813.

[2] Wei Dong, Juanru Fang, Ke Yi, Yuchao Tao, and Ashwin Machanavajjhala. 2022. R2T: Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys. In *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*. ACM, 759–772. https://doi.org/10.1145/3514221.3517844.

[3] Wei Dong, Dajun Sun, and Ke Yi. 2023. Better than Composition: How to Answer Multiple Relational Queries under Differential Privacy. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 123:1–123:26. https://doi.org/10.1145/3589268.

[4] Jianzhe Yu, Wei Dong, Juanru Fang, Dajun Sun, and Ke Yi. 2024. DOP-SQL: A General-purpose, High-utility, and Extensible Private SQL System. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4385–4388. https://doi.org/10.14778/3685800.3685881.

[5] Frank McSherry. 2009. Privacy Integrated Queries: An Extensible Platform for Privacy-preserving Data Analysis. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data*. ACM, 19–30. https://doi.org/10.1145/1559845.1559850.

[6] Noah Johnson, Joseph P. Near, and Dawn Song. 2018. Towards Practical Differential Privacy for SQL Queries. *Proceedings of the VLDB Endowment* 11, 5 (2018), 526–539. https://www.vldb.org/pvldb/vol11/p526-johnson.pdf.

[7] Ios Kotsogiannis, Saeed Meiser, Joseé M. del Alamo, Aastha Chaudhuri, Mohammad Hayyeri, Gerome Miklau, Michael Hay, and Ashwin Machanavajjhala. 2019. PrivateSQL: A Differentially Private SQL Query Engine. *Proceedings of the VLDB Endowment* 12, 11 (2019), 1371–1384. https://dl.acm.org/doi/10.14778/3342263.3342274.

[8] Transaction Processing Performance Council. 2024. TPC Benchmark H (Decision Support) Standard Specification. Technical report.